Data Science Project

(Spam Prediction on Spare Text Dataset)

By

Jabez Solomon J

Introduction:

In today's digital world, spam mail, or junk email, presents a significant and multifaceted challenge. It accounts for a considerable portion of global email traffic, consuming vast amounts of internet bandwidth and storage space, which in turn increases operational costs for service providers and businesses. Beyond the annoyance of unsolicited messages, spam poses serious security risks, often containing malicious links or attachments that can lead to malware infections and phishing attacks. This not only jeopardizes personal information security but also poses a threat to businesses' data. The time spent by employees in managing and deleting spam emails results in productivity loss, diverting attention from their core work responsibilities. Additionally, spam emails are frequently used for scams and fraudulent activities, leading to potential financial losses and identity theft. This issue also impacts legitimate email marketing efforts, as important communications might inadvertently be marked as spam, leading to communication barriers and missed business opportunities. Governments and regulatory bodies face ongoing challenges in enforcing laws like the CAN-SPAM Act in the United States, especially with spammers often operating across international borders. Finally, for individual users, the inundation of spam in inboxes not only deteriorates the email user experience but also diminishes trust in email communications as a whole. Thus, the impact of spam mail extends from individual inconvenience and security risks to broader economic and regulatory challenges, necessitating a combination of technological, legal, and educational strategies to combat its effects.

Machine learning plays a pivotal role in tackling the issue of spam mail, representing a sophisticated and adaptive approach to identifying and filtering out unwanted emails. By leveraging the capabilities of machine learning, email systems can analyze vast quantities of data, learning to differentiate between spam and legitimate messages based on patterns and characteristics identified in the content. In the context of Google Mail (Gmail), machine learning is at the heart of its spam filtering system. Gmail employs advanced algorithms that continuously learn and evolve. These algorithms analyze a range of features in emails, such as headers, the text of the message, metadata, and even the way users interact with different emails.

They look for patterns that typically indicate spam, such as certain phrases, suspicious links, or the sending behavior of the email source. One key aspect of Gmail's approach is its use of user feedback. When a user marks an email as spam or not spam, this information is fed back into the system, further refining the machine learning model. Additionally, Gmail's machine learning algorithms are tailored to adapt to the ever-changing tactics of spammers, making its spam detection not only reactive but also proactive in anticipating new spamming methods. This integration of machine learning into email filtering systems like Gmail's results in a more efficient, accurate, and user-responsive approach to combating spam, significantly enhancing the overall email experience by reducing the influx of unwanted messages and increasing the security and reliability of email communication.

Python Library:

The below mentioned python libraries are the implemented library for this project:

1. Pandas: Pandas is a foundational Python library for data analysis and manipulation. It introduces two primary data structures, DataFrame and Series, which are essential for handling and processing structured data efficiently. Pandas excels in tasks such as data cleaning, transformation, analysis, and visualization, making it an indispensable tool in data science workflows.

2. Matplotlib.pyplot: This library, a component of Matplotlib, offers a MATLAB-like plotting framework. It's widely used for creating a wide range of static, interactive, and animated visualizations in Python. Matplotlib.pyplot is highly customizable and can produce complex graphs and charts integrated with Pandas data structures.

3. Seaborn: Built on top of Matplotlib, Seaborn is a statistical data visualization library that provides a high-level interface for drawing attractive and informative statistical graphics. It is particularly effective for creating complex visualizations such as heatmaps, time series, and violin plots with ease and efficiency.

4. Joblib: Joblib is a set of tools providing lightweight pipelining in Python. It is especially useful for saving and loading large objects that contain big data, commonly encountered in machine learning environments. The library is optimized for handling objects with large arrays of data, like NumPy arrays, making it ideal for persisting machine learning models.

5. Openpyxl: Openpyxl allows for reading and writing Excel 2010 xlsx/xlsm/xltx/xltm files in Python. It enables the handling of Excel files without the need for Microsoft Excel, allowing for the automation of Excel document processing and analysis directly within a Python environment.

6. Re: Python's standard library for regular expressions, 're' is crucial for string searching and manipulation. It is widely employed in data cleaning and preparation, offering functionalities for finding, replacing, or removing text based on identified patterns.

7. Sklearn.model_selection.train_test_split: This function from the scikit-learn library is essential for splitting datasets into random train and test subsets. Such splitting is vital in machine learning for validating the performance of models, ensuring that they generalize well to new, unseen data.

8. Sklearn.naive_bayes.MultinomialNB: Part of the scikit-learn library, MultinomialNB is a classifier based on the Naive Bayes algorithm for multinomially distributed data. It is particularly suited for text classification tasks where features are typically represented as word vector counts or frequencies.

9. Sklearn.ensemble.RandomForestClassifier: This is a robust meta-estimator from scikit-learn that fits multiple decision tree classifiers on various sub-samples of the dataset. By

averaging their predictions, it enhances predictive accuracy and controls over-fitting, making it a powerful tool for a wide range of classification tasks.

10. Sklearn.metrics: This module in scikit-learn encompasses a range of metric functions like "**accuracy_score"**, "**classification_report"**, "**confusion_matrix"**, "**roc_curve"**, **auc**, and "**precision_recall_curve"**. These metrics are instrumental in evaluating the performance of classification models, providing insights into aspects such as accuracy, precision, recall, and the receiver operating characteristic (ROC) curve.

These libraries collectively form the backbone of the data processing, modeling, and evaluation stages in your machine learning project, offering a comprehensive toolkit for effective and efficient data-driven analysis and decision-making.

Data Preprocessing:

In our email analysis project, the dataset comprises a substantial volume of data with 5173 rows and 3002 columns. Each column potentially represents different attributes or words extracted from email contents. We implemented several data preprocessing steps to extract meaningful features from this extensive dataset. Here's an overview of the key preprocessing steps and their significance:

1. **Exclusion of Non-Feature Columns**: Our initial step involved identifying and excluding 'Email No.' and 'Prediction' columns from feature calculations. 'Email No.' serves as a unique identifier for each email, while 'Prediction' likely represents the outcome of a previous classification model, such as indicating spam or non-spam emails. Since these columns do not contribute to the feature set, they were excluded from further analysis.
2. **Calculation of Email Length**: We computed the 'email_length' for each email by summing the values across all columns, excluding the non-feature columns. This calculation is essential as the length of an email, determined by the frequency or presence of specific words or attributes, can be a significant feature in classifying emails, especially in identifying spam.
3. **Count of Capitalized Words**: The preprocessing included calculating the 'capitalization_count', which measures the number of words starting with a capital letter in each email, again excluding the specified non-feature columns. The frequent use of capitalized words in emails is often characteristic of promotional or spam content, making this a valuable feature in our analysis.
4. **Word Count Calculation**: An important step was the computation of 'word_count'. We counted the number of unique words in each email, considering each non-zero entry in a word column as one word. This feature is useful in assessing the complexity and informativeness of the email content.

These preprocessing steps, namely calculating email length, capitalization count, and word count, form a crucial part of our comprehensive data preprocessing strategy. They enable us to transform the raw email data into a structured format that is more conducive to analysis and modeling. It's important to note that these steps are just a segment of the broader data

preprocessing efforts undertaken in our project. Further preprocessing steps may include normalization, handling missing values, and feature selection, which are essential to prepare the dataset for effective machine learning model implementation.

Spam vs Non-Spam Distribution

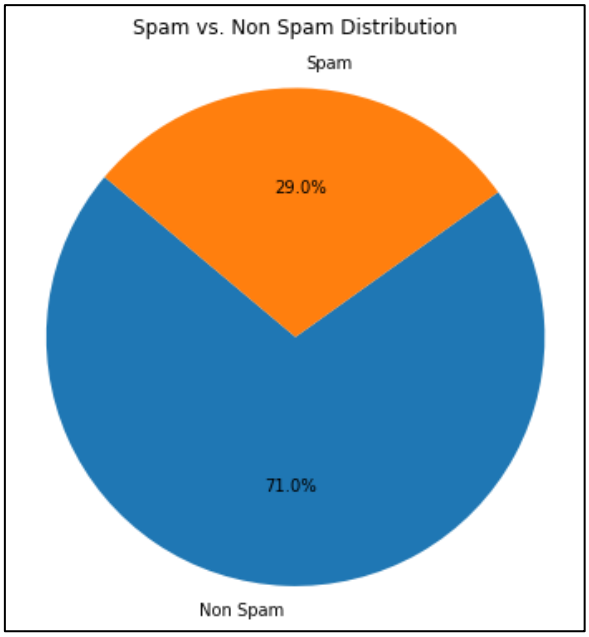


Fig 1: Spam vs Non-Spam Distribution

A critical step in our data preprocessing was the analysis of class distribution within the 'Prediction' column of our dataset, where the value '0' denotes Non-Spam and the value '1' denotes Spam. The pie chart, titled "Spam vs. Non-Spam Distribution," illustrates this distribution and reveals a significant imbalance: 71% of the emails are classified as Non-Spam, while only 29% are classified as Spam.

This imbalance in class distribution presents a notable challenge for training machine learning models. Models trained on imbalanced data can develop a bias towards the majority class—in this case, Non-Spam—resulting in less accurate detection of Spam emails. It is crucial for the success of our machine learning endeavors to address this imbalance. Techniques such as resampling the dataset, using anomaly detection algorithms, or applying different weights to the classes during model training may be considered to mitigate the issue and improve the model's ability to generalize across both classes.

The pie chart underscores the importance of ensuring our machine learning algorithms are trained on a dataset that is as representative as possible of the true distribution of classes in the real world. It also serves as a visual reminder to prioritize the refinement of our preprocessing techniques to enhance model performance despite the challenges posed by imbalanced data.

Spam vs Non-Spam words Comparison:

An integral part of our data analysis involved examining the most frequently used words in spam and non-spam emails. The accompanying visualizations, generated from our dataset,

highlight the top fifteen words in each category, providing insight into the lexical patterns that may help distinguish between spam and legitimate correspondence.

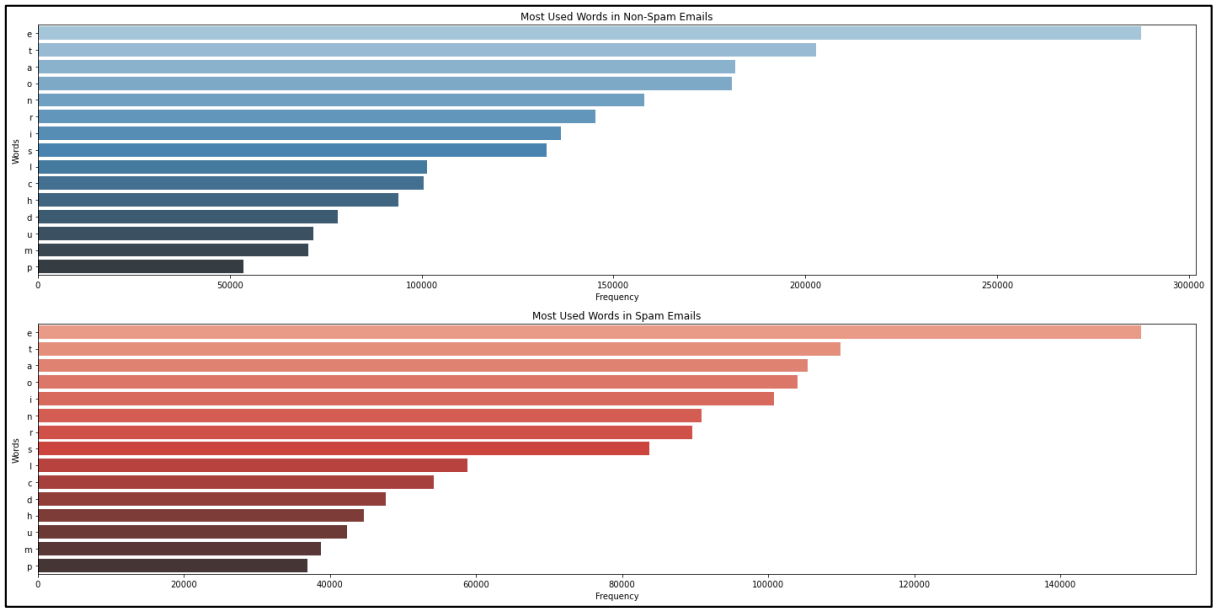


Fig 2: Most Used Words in Spam and Non Spam Mails

The bar graph titled "Most Used Words in Non-Spam Emails," depicted in shades of blue, shows a higher frequency of words that are typical in business or formal communication. This distribution suggests a vocabulary associated with routine, professional, or possibly transactional language.

Conversely, the "Most Used Words in Spam Emails" graph, presented in shades of red, reveals a distinct set of words that are prevalent in spam. These terms may indicate a focus on urgency, action, or sales, which are common characteristics of unsolicited promotional content.

To generate these insights, we processed our email dataset by separating it based on the 'Prediction' column, where '0' indicates non-spam and '1' indicates spam. We then excluded non-word columns, such as 'Email No.', 'Prediction', 'email_length', 'capitalization_count', and 'word_count', to focus solely on the actual content of the emails. Afterward, we calculated the sum of occurrences for each word in both non-spam and spam categories and sorted them in descending order to identify the most prevalent terms.

The resultant graphs were plotted using Seaborn's barplot function, providing a clear visual contrast between the two types of emails. The x-axis represents the frequency of each word, while the y-axis lists the words themselves.

To better understand the distribution of these “**email_length”**, “**capitalization_count”**, and “**word_count**”, we categorized the data into bins and visualized their distributions.

The 'Email Length Distribution' graph, presented in blue, shows that the majority of emails fall within the 101-500 and 501-1000 character ranges, with a significant drop as the character count increases. This suggests that most emails are concise, with fewer instances of extremely long emails in our dataset.

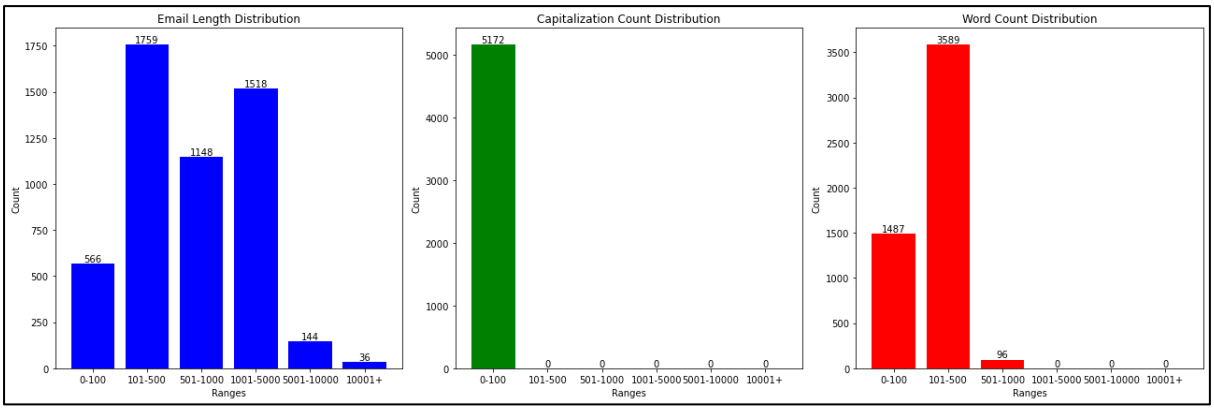


Fig 3: Distribution Graph

The 'Capitalization Count Distribution' graph, illustrated in green, highlights that a vast majority of emails have a capitalization count between 0-100, indicating a standard use of capitalization in the corpus. Very few emails contain an excessive number of capitalized words, which might be characteristic of promotional content or spam.

The 'Word Count Distribution' graph, depicted in red, indicates that most emails have a word count in the 101-500 range, followed by the 0-100 range. Similar to email length, there's a noticeable decline in frequency as the word count increases, showing that the dataset contains predominantly shorter emails.

We implemented data binning to segment these attributes into meaningful ranges using custom bins defined as **[0, 100, 500, 1000, 5000, 10000, max_value]**. Each email was then categorized based on these bins for the three attributes. We utilized Matplotlib to create bar graphs for a visual representation of the distribution within these bins. The count for each bin is annotated on the graphs, facilitating an immediate understanding of the distribution.

Machine Learning

Upon completing the preprocessing of our email dataset, we proceeded to the model training and testing phase to identify the most effective algorithm for our classification task. We chose two widely recognized machine learning models for our comparative analysis: Naive Bayes and Random Forest.

Naïve Bayes:

The Naive Bayes classifier is particularly well-suited for text classification due to its simplicity, efficiency, and the probabilistic foundation that handles a large feature space effectively. Given the nature of our email data, which involves high-dimensional text data, Naive Bayes offers a strong baseline due to its assumption of feature independence and its ability to handle both binary and multi-class classification problems. Its efficiency in processing and its robustness to irrelevant features make it a compelling choice for our initial model training.

```
Model Accuracy: 0.9355670103092784

Performance Report:
              precision    recall  f1-score   support

           0       0.95      0.95      0.95      1097
           1       0.89      0.89      0.89       455

    accuracy                           0.94      1552
   macro avg       0.92      0.92      0.92      1552
weighted avg       0.94      0.94      0.94      1552
```

Fig 4: Naïve Baise Accuracy Score

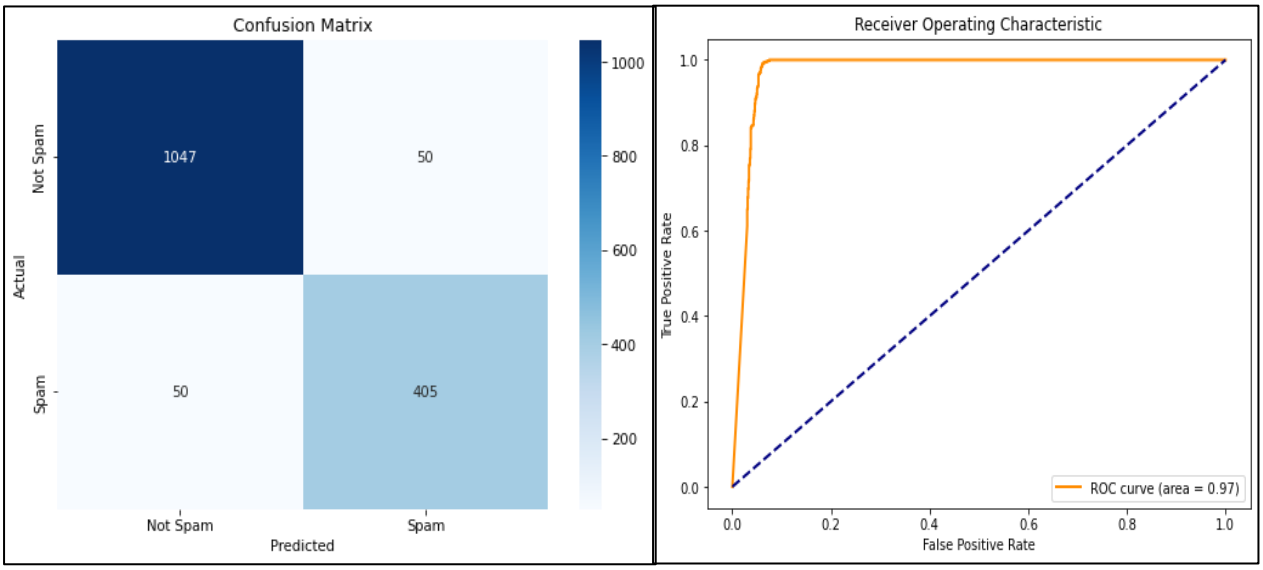


Fig5: Naïve Baise Confusion Matrix and ROC

Our email classification model, based on the Naive Bayes algorithm, has undergone a thorough evaluation. The performance metrics are highly encouraging, demonstrating the model's robustness in distinguishing between spam and non-spam emails.

The model has achieved an accuracy of approximately 93.56%, which indicates a high level of precision in classification tasks. This metric reflects the overall ability of the model to correctly identify spam and non-spam emails out of all classification decisions made.

The detailed performance report shows precision, recall, and F1-score for both classes:

- For Non-Spam (labelled as '0'), the model shows a precision and recall of 0.95, which implies that 95% of the emails predicted as non-spam were correct, and the model correctly identified 95% of all actual non-spam emails.
- For Spam (labelled as '1'), the model achieved a precision and recall of 0.89, suggesting that while slightly less precise and sensitive than with non-spam, it still performs well in identifying spam emails.

The F1-score, which combines precision and recall into a single metric, stands at 0.95 and 0.89 for non-spam and spam respectively, indicating excellent performance for both categories.

The ROC curve further cements the model's efficacy, boasting an area under the curve (AUC) of 0.97. This near-perfect AUC score suggests that the model has an exceptional ability to differentiate between the spam and non-spam classes, with a high true positive rate and a low false positive rate.

The confusion matrix provides a visual and quantitative depiction of the model's performance. Out of the test set, 1,047 non-spam emails were correctly classified with only 50 false positives. Similarly, 405 spam emails were correctly identified with the same number of false negatives. This indicates a balanced performance in identifying both classes, despite the potential challenges posed by imbalanced data.

The results of the Naive Bayes classifier are promising, demonstrating that it is a viable tool for spam email classification. Its performance is characterized by high accuracy and a commendable balance between precision and recall across both email categories.

Random Forest:

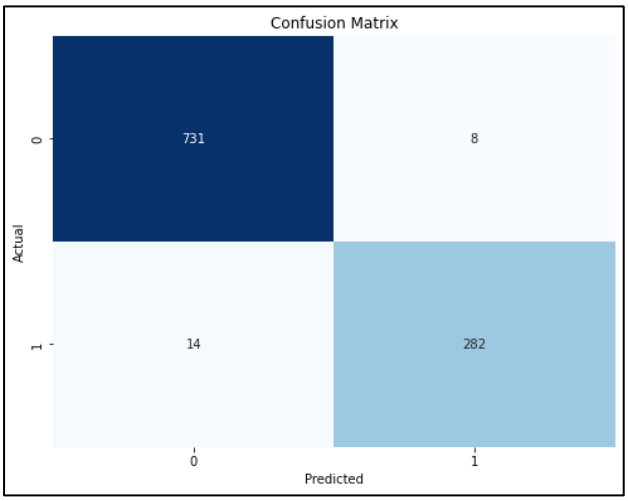
In contrast, the Random Forest classifier is an ensemble learning method known for its high accuracy and robustness. By aggregating multiple decision trees to determine the final output, Random Forest reduces the risk of overfitting—a common concern in complex models. It is particularly adept at handling datasets with imbalanced classes, as in our case, where spam emails are less frequent than non-spam. Furthermore, Random Forest's ability to rank the importance of features provides valuable insights, which is instrumental in understanding the underlying patterns in spam detection.

Accuracy Score: 0.978743961352657

Performance Report:

	precision	recall	f1-score	support
0	0.98	0.99	0.99	739
1	0.97	0.95	0.96	296
accuracy			0.98	1035
macro avg	0.98	0.97	0.97	1035
weighted avg	0.98	0.98	0.98	1035

Fig 6: Random Forest Accuracy Score



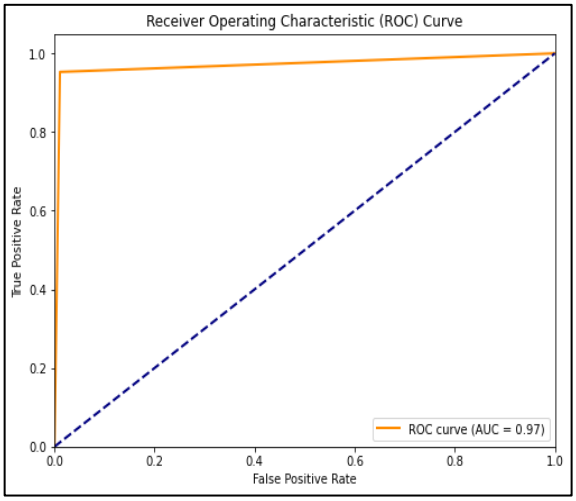


Fig 7: Random Forest Confusion Matrix and ROC

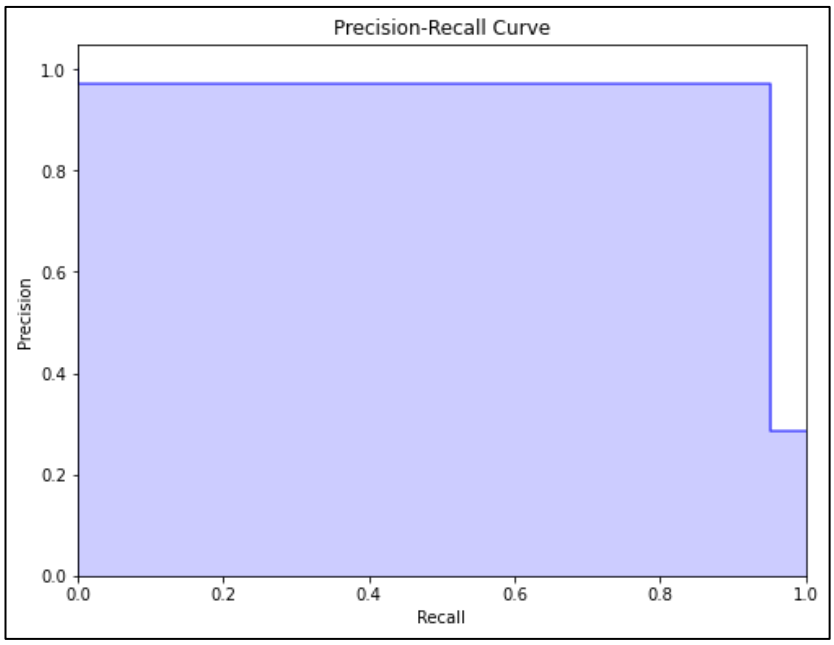


Fig 8: Random Forest Precision Recall Curve

Following the Naive Bayes classifier, we applied the Random Forest algorithm to our preprocessed email dataset and conducted a performance evaluation. The results indicate that the Random Forest classifier has achieved a high degree of accuracy and precision.

The accuracy score for the Random Forest model is approximately 97.87%, a marked improvement over the Naive Bayes classifier. This score indicates an excellent overall correct classification rate for both spam and non-spam emails.

The precision, recall, and F1-scores are as follows:

- Non-Spam (labelled '0') received a precision of 0.98 and a recall of 0.99, with an F1-score of 0.99. This indicates that the model is extremely precise and reliable in identifying non-spam emails.
- Spam emails (labelled '1') had a precision of 0.97 and a recall of 0.95, with an F1-score of 0.96, suggesting that the model also accurately identifies the majority of spam emails with very few misclassifications.

The precision-recall curve demonstrates the trade-off between precision and recall for the spam class (labelled '1'). The model exhibits high precision across all levels of recall, which is ideal for a spam detection system where the cost of false positives is high.

The ROC curve for the Random Forest model shows an AUC of 0.97, identical to the Naive Bayes model, affirming its excellent classification capabilities.

The confusion matrix displays that out of the test data, 731 non-spam emails were correctly classified with only 8 false positives. In the case of spam emails, 282 were correctly classified with 14 false negatives. These results underscore the model's balanced accuracy in identifying both classes.

Comparison Conclusion:

When comparing the performance of the Naive Bayes and Random Forest classifiers, the Random Forest model demonstrates superior performance metrics across accuracy, precision, recall, and F1-score. Although both models show high AUC values, suggesting excellent classification abilities, the Random Forest model has the edge due to its higher precision and recall, particularly in the classification of spam emails.

The Random Forest classifier's ability to handle the complexity and potential imbalances in the dataset, combined with its interpretability through feature importance scores, makes it the preferred model for our email classification task. It has not only achieved higher accuracy but also shown greater reliability in distinguishing between spam and non-spam emails.

Machine Learning Performance Boost:

In pursuit of enhanced accuracy for our machine learning models, we employed a strategic method of dataset refinement based on the distribution of email_length from Fig 3 Distribution Graph. Recognizing that certain ranges of email length may have a more significant impact on classification performance, we focused on the bins that represent the most common lengths of emails: '101-500', '501-1000', and '1001-5000'. These bins were chosen based on their higher frequency in the dataset, which suggests a potential correlation with the predictive features of our model.

We filtered our main dataset to include only those records where the "**email_length"** falls within the aforementioned bins. This selective filtering is hypothesized to concentrate the dataset on more characteristic email lengths, thereby excluding outlier lengths that could potentially skew the model's learning process.

After applying the filter, the refined subset of the dataset was exported to an Excel file, aptly named **'filtered_data.xlsx'**. This ensures that our refined dataset is preserved for reproducibility and further analysis, allowing us to trace back our steps in the data preprocessing stage.

After refining our dataset to focus on emails within the most common length bins, we re-evaluated our machine learning models. The intent was to observe any performance improvements by concentrating on more typical email lengths that could potentially contain more predictive features for spam detection.

Updated Naïve Baise:

The refined Naive Bayes model exhibits a strong accuracy score of approximately 95.33%, indicating a high level of precision in its classification decisions.

The precision, recall, and F1-scores are reflective of the model's performance:

- For Non-Spam (labelled '0'), the model achieved a precision of 1.00, indicating no false positives, and a recall of 0.94, with a resulting F1-score of 0.97.
- For Spam (labelled '1'), the model achieved a precision of 0.86 and a recall of 1.00, indicating no false negatives, with an F1-score of 0.93.

```
Model Accuracy: 0.9533132530120482

Performance Report:
              precision    recall  f1-score   support

           0       1.00      0.94      0.97       940
           1       0.86      1.00      0.93       388

    accuracy                           0.95      1328
   macro avg       0.93      0.97      0.95      1328
weighted avg       0.96      0.95      0.95      1328
```

Fig 9: Updated NB Model Accuracy

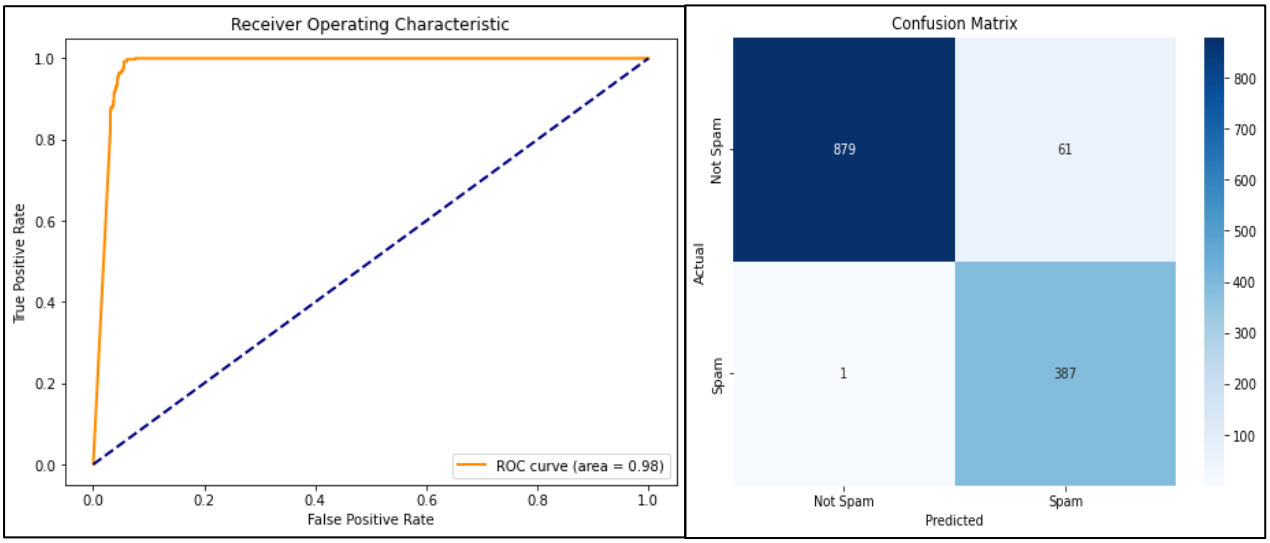


Fig 10: Updated NB Model ROC and Confusion Matrix

The ROC curve maintains an impressive AUC of 0.98, reaffirming the model's exceptional capability to distinguish between spam and non-spam emails effectively.

The updated confusion matrix shows a high number of true positives and true negatives, with 879 correctly classified non-spam emails and 387 spam emails correctly identified. The numbers of false positives and false negatives are minimal, suggesting that the refined model has retained its discernment abilities well.

Updated Random Forest:

The Random Forest model showcased an accuracy score of approximately 96.94%. This high accuracy rate is indicative of the model's proficient classification capabilities within the context of the refined dataset.

The performance metrics are detailed as follows:

- The curve indicates that the model maintains a commendable balance between precision and recall across the spectrum, essential for a spam detection system where the cost of misclassification can be significant.
- The ROC curve presents an AUC of 0.96, slightly lower than the previous iteration, yet still reflecting a strong discriminative ability between the classes.

Accuracy Score: 0.9694915254237289

Performance Report:

	precision	recall	f1-score	support
0	0.97	0.99	0.98	613
1	0.97	0.93	0.95	272
accuracy			0.97	885
macro avg	0.97	0.96	0.96	885
weighted avg	0.97	0.97	0.97	885

Fig 11: Updated Random Forest Model Accuracy

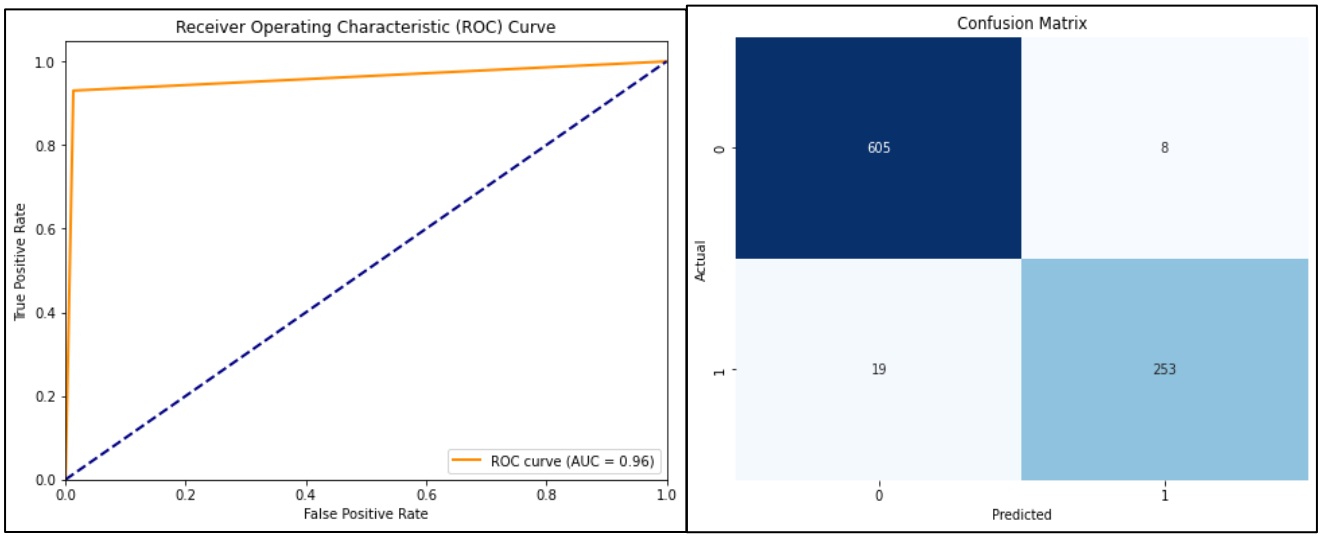


Fig 12: Updated Random Forest Model's ROC and Confusion Matrix

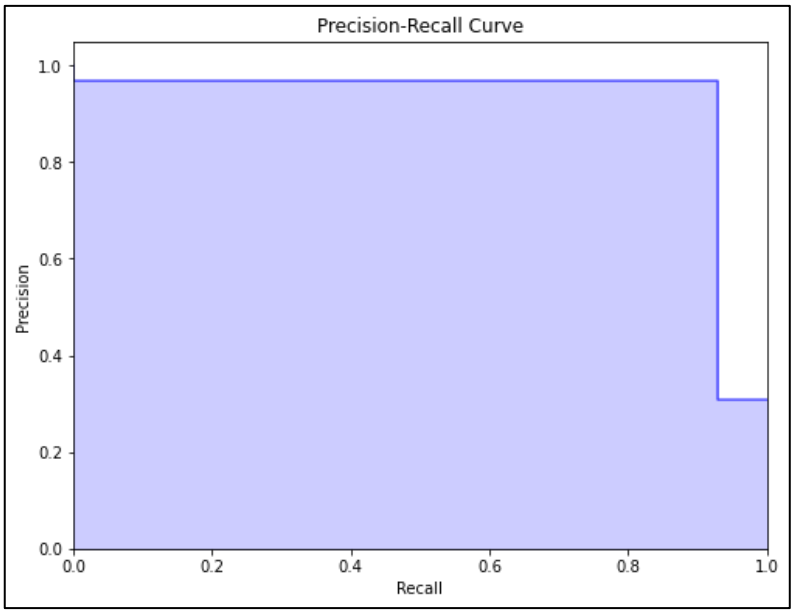


Fig 13: Random Forest Precision Recall Curve

The updated matrix shows that the model accurately classified 605 Non-Spam emails while correctly identifying 253 Spam emails. The number of false positives and false negatives (8

and 19, respectively) indicates that the model is quite reliable, though there is a minor increase in false negatives.

The refined Random Forest classifier demonstrates a slight reduction in performance compared to the previous results, with a minor decrease in recall for Spam emails and a slight decrease in the AUC. Nevertheless, the high accuracy and F1-scores across both Non-Spam and Spam classifications suggest that the model remains robust and effective.

Updated Conclusion:

In summary, the refinement of the dataset to include emails within specific length ranges did not significantly enhance the Random Forest classifier's performance. Both Naive Bayes and Random Forest classifiers performed comparably well, with each showing its respective strengths. The Random Forest classifier, while exhibiting a marginal decrease in certain metrics, continues to be a strong model for the task of email classification.

Deploying the Model:

As of now we know through the entire machine learning process, that Random Forest was the best, that model has be trained and saved into **'spam_classification_model.joblib'.**

Spam Classification Web Application Development:

In order to operationalize our spam classification model, we developed a web application using Streamlit. This application provides a user-friendly interface for uploading text files and utilizes our pre-trained Random Forest model to predict whether an email is spam.

Functionality Overview:

- The application includes a sidebar for file upload, specifically designed to accept "**.txt"** files, which are presumed to contain email content.
- Upon upload, the application reads and cleans the text data, extracting words and removing non-alphabetic characters. It then compares these words with the column names from a pre-existing Excel template that represents our feature set.
- A button titled 'Process Text File' triggers the extraction and comparison process. It also initiates the loading of the pre-trained Random Forest model, which is stored in a "**.joblib"** file, and makes predictions based on the processed text file.
- The predictions, along with the original data, are updated in a new Excel file, which is then made available for download directly from the application's sidebar.
- The main content area of the application displays the predictions, distinguishing spam emails from non-spam ones.

Technical Implementation:

- The "**extract_words_from_txt"** function handles the reading and preprocessing of the text file.
- The "**compare_and_update_excel"** function aligns the extracted words with the feature set and updates a new row in the Excel template, preparing the data for prediction.
- The "**load_and_predict_model"** function loads the stored Random Forest model and performs the classification task.
- The application ensures that the entire process, from file upload to data processing, model prediction, and file download, is seamless and interactive.

Deployment and Usage:

- The web application is designed for ease of use, requiring users to simply upload a text file to receive instant spam classification results.
- Once the text file is processed, the results are displayed in the application, and the updated Excel file with the predictions is available for download.
- A summary of the classification is provided in the main content area, offering clarity on the classification results **"(1 for spam and 0 for not spam)".**

Conclusion:

This Streamlit-based web application signifies a practical step towards the real-world application of our spam classification model. It encapsulates the functionality of our Random Forest classifier within a straightforward and accessible interface, allowing for immediate spam detection and classification without the need for in-depth technical knowledge.

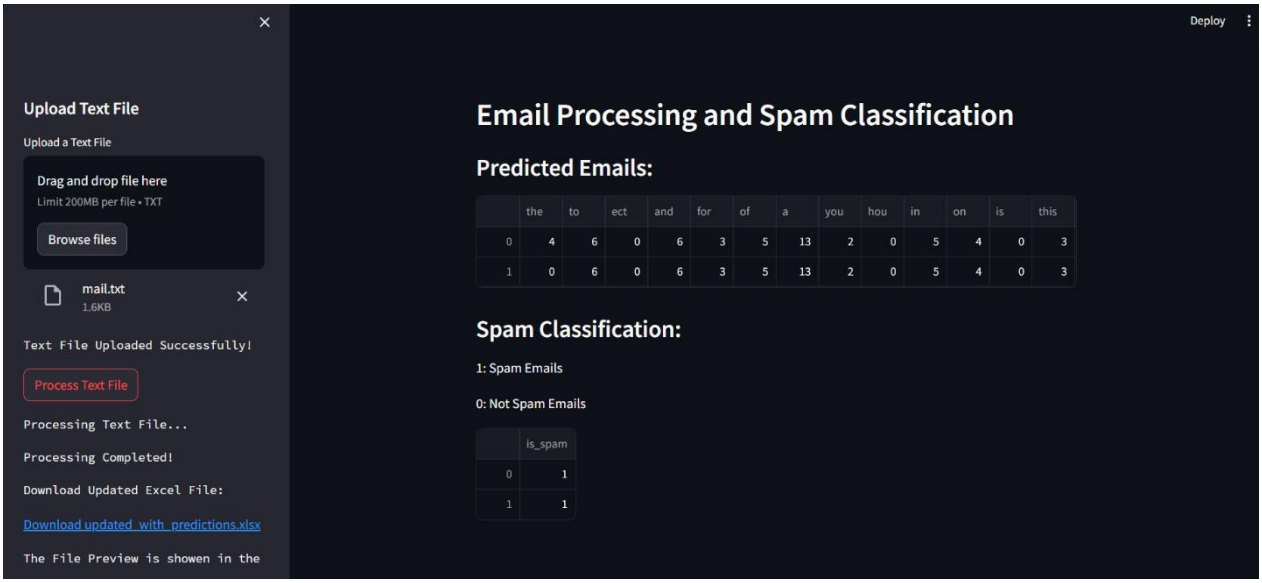


Fig 14: Deployment Final Output